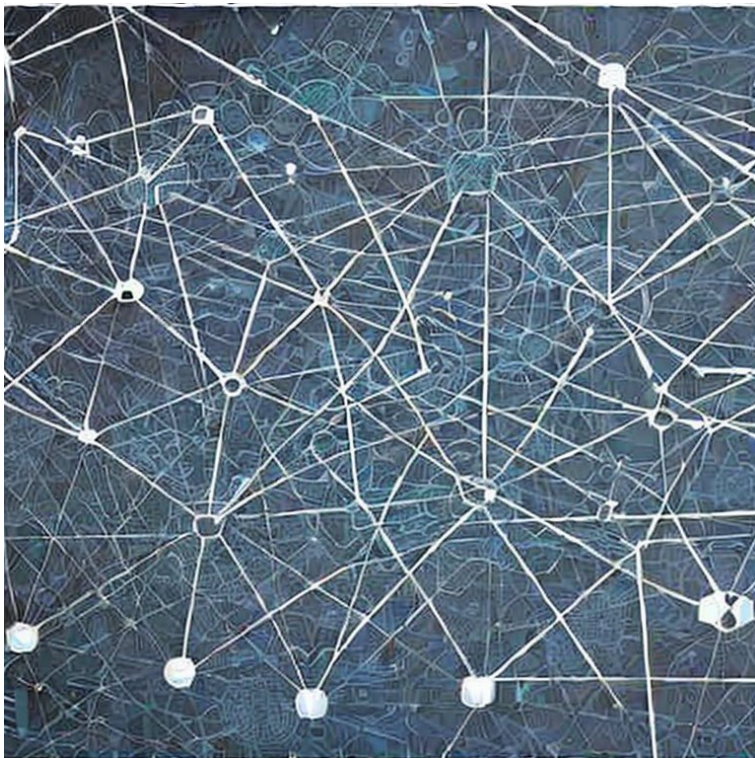




Master's thesis

Monitoring Safety Properties in Event-Driven Microservices

Adam Zdziechowski
Klaus Philipp Theyssen

Advisors: Yongluan Zhou, Rodrigo Laigner

Submitted: May 31, 2023

Abstract

Large parts of our modern society depend on software, therefore creating dependable software artifacts is an important problem.

At the time of writing microservice architectures are a widespread approach for constructing software systems. These architectures promise several benefits like scalability, fault tolerance, improved resource utilization in the cloud and an improved development process across multiple engineering teams. At the same time implementing the microservices architecture comes with several challenges and potential pitfalls that naturally arise in distributed systems. In this thesis, we are concerned with a certain class of microservices, namely event-driven architectures, in which the individual components communicate via asynchronous events using a messaging system.

An inherent challenge in these systems is dealing with data integrity, as each service typically uses its own data store. Due to a lack of a centralized component that enforces constraints, one might experience faulty replication and inconsistencies in the data.

To address this issue we created a simulation tool for microservice applications with the goal of quantifying data integrity violations. For this, we have created an invariant definition language and a workload generator to execute specific workloads against the system under test. The envisioned use by a developer is to define key invariants about an application and a workload scenario. The tool translates the invariants into stream processing operators which monitor the invariants while executing the workload. The resulting metrics are visualized and presented to the user in a dashboard. The tool aims to allow fast experimental investigations of the data integrity properties of the application. This in turn lets the developer make informed decisions about how to evolve an application architecture, to ensure costly anomalies are discovered before the application is deployed into the production environment.

Contents

Abstract	2
Contents	3
1 Introduction	5
1.1 Motivation	5
1.2 Contribution	6
1.3 Structure of the Thesis	6
2 Background	8
2.1 Microservices	8
2.1.1 Challenges of Microservice Architecture	9
2.2 Event-Driven Architectures	9
2.2.1 Online Event Processing	9
2.2.2 Complex Event Processing	9
2.3 Data Management in Microservices	10
2.4 Software Correctness	10
3 Invariants for Event-Driven Microservices	12
3.1 Exploring Invariants	12
3.1.1 Invariants for eShopOnContainers	12
3.2 Invariant Language	17
3.2.1 Stateless Invariants	17
3.2.2 Stateful Invariants	19
3.3 Expressibility	20
3.4 Invariants Revisited	20
3.4.1 EshopOnContainers Invariants	20
3.4.2 Lakeside Mutual Invariants	25
4 Monitoring Event-Driven Microservices	27
4.1 Overall Architecture	27
4.2 Workload Generator	27
4.2.1 Workload Definition	28
4.2.2 Design and Implementation	31

<i>CONTENTS</i>	4
4.3 Invariant Translator	32
4.3.1 Design and Implementation	32
5 Demonstration	36
5.1 Example Application: eShopOnContainers	36
5.2 Preliminary assessment	37
5.3 Workload	37
5.4 Specifying the invariants	37
5.5 Results	38
6 Related Work	43
6.1 HawkEDA	43
6.2 Preventing Violations	43
6.3 SQL Match Recognize Construct	44
6.4 Runtime Monitoring	44
7 Conclusion	47
7.1 Future Work	47
Bibliography	49
A Invariant Language Grammar	53
B eShop I2 - Most Recent Price in Java	56
C eShop I1 - Overselling Items	58
D Monitoring Dashboard	60

Chapter 1

Introduction

1.1 Motivation

Reasoning about the correctness of distributed systems is notoriously difficult due to their sheer complexity. At the same time microservices, a fundamentally distributed approach to building software systems (see section 2.1), are becoming prevalent in industry. This puts an extra burden on developers that create such applications. Especially data management code is a frequent source of problems, as practitioners in the industry often come up with their custom solutions containing subtle errors (section 2.3).

To illustrate this we imagine a developer, let's call her Alice, working in e-commerce. She and her team are faced with the challenge of building a scalable system that can efficiently handle a wide range of different workloads. Naturally, an online store can experience relatively low traffic at some times and peak traffic during events like Black Friday. At the same time ensuring correctness and flawless processing of payments is crucial, as an outage of the system or data inconsistencies would potentially result in huge economic losses.

The team decided to follow an event-driven architectural style (section 2.2), in which microservices communicate only via asynchronous events. Following best practices the team surrounding Alice already has a monitoring solution like Prometheus¹ in place and additionally uses a distributed tracing tool like Jaeger² to improve the observability of their application. This allows them to gather metrics, and investigate potential performance degradation.

According to the monitoring services the application is healthy and working. Nevertheless, Alice knows that eventual consistency may cause some problems when checking out items as the item is replicated in data stores across multiple

¹<https://prometheus.io/>

²<https://www.jaegertracing.io/>

microservices. She wants to investigate potential problems involving events the services use to communicate.

Existing query languages of monitoring solutions for example PromQL or Kibana Query Language (KQL) are based on SQL and regular expressions, which make them unfit for Alice’s problem. They make it hard to directly express causal and temporal relationships about events and even impossible to reason about the absence of events.

As the team is dealing with streams of events, using a stream-processing framework seems like an obvious choice for monitoring more complicated properties or invariants of the application. Unfortunately, no one in the team has prior experience with such frameworks and ideally, the developers want to focus on building their application and not on writing complicated stream processing operators. They know which invariants they care about and simply need a way to express them.

In section 5.1 we visit this example again and show how our tool might help the developers.

1.2 Contribution

We developed a tool (chapter 4) that allows the user to specify invariants using a custom DSL and a specific workload using a configuration language. The tool monitors the invariants while executing the workload against the microservice application. The invariants defined by the user are translated into Apache Flink jobs, enabling real-time monitoring of the invariants. This makes it possible to quickly experiment with different workloads and invariants in order to discover potential bugs in the application.

Further, in chapter 5 we perform a demonstration of our tool on the open-source microservice application `eshopOnContainers`[1]. This showcases how to create a realistic workload and how to come up with relevant invariants to discover problematic parts of the application.

1.3 Structure of the Thesis

The remainder of this thesis is structured as follows:

Chapter 2: We review fundamental concepts surrounding microservice application and event-driven architectures. We also discuss software correctness in general including the concept of invariants.

Chapter 3: We explore invariants in event-driven microservice applications. Next, we describe our design process for coming up with a DSL for defining

them. Finally, we express various example invariants in our definition language.

Chapter 4: We describe the overall architecture and components of the tool we developed. We discuss design choices and implementation details.

Chapter 5: We apply the tool to an example microservice application, showcasing how a developer might use it. We show how to choose invariants, how to write an example workload, and discuss visualizing the results in a dashboard.

Chapter 6: This chapter reviews previous and related work, with a focus on exploring different ways to express invariants.

Chapter 7: Reflection on our own work and outlook on further research directions.

Chapter 2

Background

2.1 Microservices

A popular approach to overcoming tight coupling in applications is the microservices design pattern. Microservices are designed as self-contained and independent applications. They follow the UNIX philosophy of doing a single thing and doing it well. More complex applications can be built up by connecting several microservices over standardized interfaces such as HTTP or via asynchronous events. Since microservices are decoupled from each other and communicate over well-defined interfaces, each microservice can be implemented with a different programming language and use a different data store.

Some of the benefits of adopting microservices are [2])

- higher scalability, fine-grained scaling of individual services
- robustness against failures and errors and better isolation of each component.
- microservices can be maintained, tested, and deployed individually.
- separating the central database into a database per microservice increases flexibility and reduces the need for coordination across teams when evolving the schema

Large internet companies like Amazon, Netflix, and Google have adopted this pattern for quite some time now, as especially in these huge development teams the benefits seem to outweigh the downsides. But also in general this architectural style continues to see adoption in the industry. Distributed services are omnipresent at software companies nowadays [3].

2.1.1 Challenges of Microservice Architecture

After a decade of growth in popularity and adoption in the industry, there seems to be an emerging trend among developers showing reluctance towards using distributed architectures like Microservices. Indeed, a recent article [4] from Amazon's tech blog has been making waves during the writing of this thesis.

The issues [5, 6, 3] with adopting microservices cannot be overlooked. Some of the popular complaints seem to revolve around dealing with eventual consistency guarantees when using database-per-service pattern [2]. The lack of modern infrastructure necessary to run microservices efficiently, and a relatively low level of expertise among developers, are also mentioned.

While a shift in the sentiment around microservices has been noticeable, it's most likely that with an existing high level of adoption and stronger-than-ever backing from many technological giants [7] the microservices are here to stay.

2.2 Event-Driven Architectures

Event-driven architecture (EDA) is a popular paradigm in software engineering, with some sources citing even up to 72% adoption among companies [8].

EDA promotes loose coupling between services, lending itself perfectly to a preferred medium of communication between microservices, with Apache Kafka being the most common choice for a message broker [2].

2.2.1 Online Event Processing

Different styles of EDAs have been suggested, a common one being Online Event Processing (OLEP) [9]. It offers high scalability, functional partitioning, and improved isolation but at the same time makes it impossible to enforce data integrity constraints that can otherwise be achieved with monolithic architectures[2].

This puts more burden on the developer trying to reason about correctness and the degree of consistency of the application. There is a need to address various problems like replication hell, foreign key constraints, and causal event guarantees [5].

2.2.2 Complex Event Processing

As we are dealing with event-driven architectures, the field of Complex Event Processing (CEP) [10] has to be considered.

In systems using event streaming, CEP allows for detecting patterns among said events in real time. Applications of Complex Event Processing can be

found in different domains, from detecting trends in stock prices to monitoring and alerting in healthcare devices [11].

Our idea in this thesis is to use Complex Event Processing for detecting patterns of data inconsistencies in distributed microservices applications.

CEP vendors

Many CEP solutions exist on the market, some even backed by well-established technological companies (Apache Flink CEP, Microsoft Azure Stream Analytics, Oracle CEP). Despite this, there is a lack of a clear market-share winner in this space at the moment.

We base our solution on Apache Flink CEP, due to its great integration with Apache Kafka and the maturity of Apache Flink.

2.3 Data Management in Microservices

For teams who adopt microservices applications using separate (often heterogeneous) databases, data management issues are some of the biggest challenges they face [5].

For example, in a web shop application, if an item gets removed from the Catalog database, the update should be immediately propagated to the Basket database to avoid orders being placed with non-existent products. This type of foreign key constraint tends to be difficult to apply with microservices - the developers often end up either ignoring the risks or creating sub-optimal fixes at the application code level [5].

We suggest a pragmatic approach to this - while the proper solutions are needed, we believe that better observability of data integrity violations would be a good start for any developer wanting to improve their solution.

2.4 Software Correctness

In an ideal world software development could look like a linear process:

informal requirements $\rightarrow$ formal specification $\rightarrow$ correct implementation

Achieving the second step by reasoning about programs in terms of pre- and post-conditions and invariants have been well-studied since [12].

This means the basic tools for proving programs correct have been known for a long time, nevertheless, due to economic and time constraints, most software projects are not proven correct. Although advances in formal methods make it

feasible to verify larger and larger projects [13, 14], they are still only applied to specialized software, making up a tiny fraction of all software written.

Especially for architectures like microservices, which encourage software heterogeneity, one has to resort to more lax methodologies. In such a situation, one typically resorts to testing and monitoring to increase trust in the correctness of a system.

In our context of event-driven architectures, invariants will be concerned with events, their values, temporal relationships (ordering) among them, and state (materialized views) across multiple events.

The goal is to quantify the degree to which invariants are violated. Some violations are to be expected - this comes from the fact that by adopting eventual consistency through asynchronous event-based communication strong guarantees (ACID) are abandoned. This is done consciously in favor of other desirable properties like scalability. Still, the question remains if the trade-off is worth it, i.e. how often violations are encountered.

For this, we need a way to express the invariants we have identified, which brings us to the next chapter.

Chapter 3

Invariants for Event-Driven Microservices

3.1 Exploring Invariants

The overall goal of this thesis is to monitor safety properties in event-driven architectures. We focus especially on invariants concerning data integrity.

Given a concrete application, a necessary first step is identifying such properties or invariants. For this, we considered two open-source projects `eshopOnContainers` [1] an online store in the e-commerce domain and `Lakeside Mutual` [15] an insurance application. Both of these follow an event-driven microservice architecture.

The applications are educational projects and lack some complexity that would arise in the real world. Nevertheless, they are sufficient for our demonstration purpose and at the same time, we can avoid getting lost in domain-specific details.

We now give a short introduction to both applications and formulate some example invariants in natural language for both of them. This step would usually be done by the developers, specialized analysts (e.g. requirements engineers) or other domain experts.

3.1.1 Invariants for `eShopOnContainers`

The `eShopOnContainers` application is an online shop, where users can browse different items from a catalog, put items in their basket and finally checkout, thus completing an order. The application is composed of several microservices `basket`, `catalog`, `ordering`, and `payment`. Each service uses its data-store and communication across services is done via an asynchronous message bus. In figure 3.1 we depict a simplified overview of the different components, the

actual application contains further components and a front end which is not relevant to our discussion.

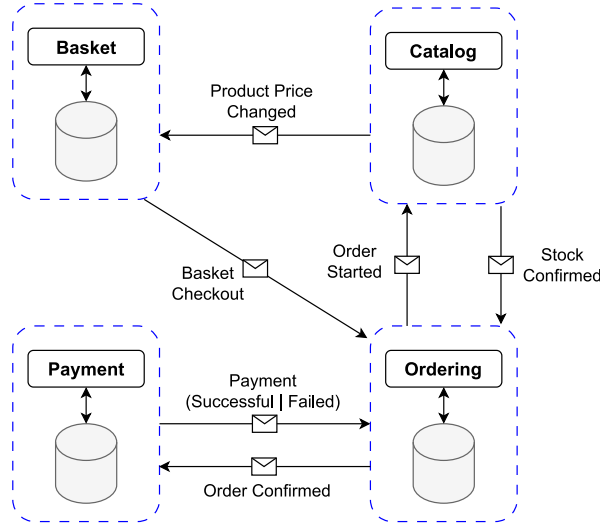


Figure 3.1: eShop components and essential events exchanged between them

For eShop we came up with the following invariants about the communication across microservices:

eShop I1 - Overselling Items: Each item in the catalog has an associated stock amount, this stock should never be negative meaning an item should never be oversold.

eShop I2 - Most Recent Price: The price of items in the catalog can be updated by creating a `PriceUpdate` event, therefore when checking out a basket all items should have the most recent price.

eShop I3 - Timely Payment: After an order has been submitted, it should be followed by either a `PaymentSucceeded` or a `PaymentFailed` event in a reasonable time (e.g. 5 min).

eShop I4 - Deleting Items from Basket: Deleting an item in the Catalog should delete it from every basket.

eShop I5 - Unsuccessful payment: Orders should not be shipped if `PaymentFailed` event has been observed.

eShop I6 - Order Creation: No duplicate `OrderStarted` events should be created, they should have a unique ID.

eShop I7 - Payment Amount Matches Product Prices: The amount paid by the customer should match up with the sum of all individual prices of the items before the order payment is considered successful.

Invariants for Lakeside Mutual

Lakeside Mutual is an insurance application, it serves the employees of the insurance company and provide services to the customers. Customers can request insurance quotes, after receiving a quote a customer can decide to accept or reject the offer. The invariants mostly revolve around actions between employees and customers.

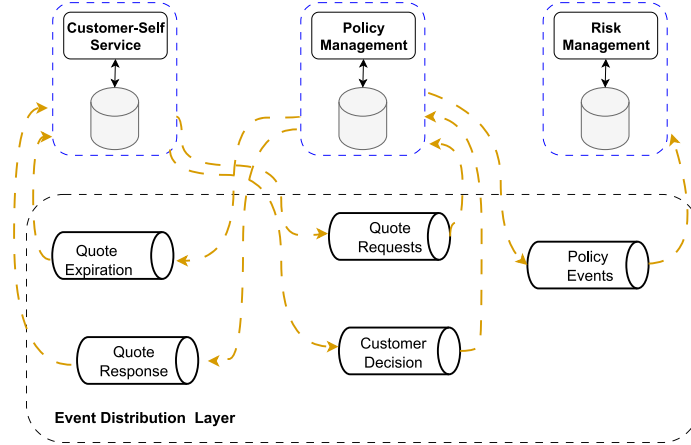


Figure 3.2: Lakeside Mutual components and events exchanged between them.

In this application possible invariants are:

Lakeside I1 - Quote expiration: A `QuoteExpiration` event should not be followed by a `CustomerDecision` event.

Lakeside I2 - Ghost Updates: After a policy is deleted there should be no changes to that policy's state.

Lakeside I3 - Timely Creation: After a customer accepts a quote a policy should be created in a timely manner.

Lakeside I4 - Timely Deletion: After a user has deleted their account, all their policies have to be deleted in a timely manner.

Monitoring Invariants with Flink CEP

After formulating these invariants in natural language, we investigated different approaches for checking that the invariants are not violated. Since we deal with continuous streams of data produced by the applications and want to get instant notifications of violations, the apparent choice is the stream-processing paradigm [16].

As the concrete framework, we choose Apache Flink [17], due to its maturity and widespread use in the industry. Flink allows building stream queries,

consisting of fundamental constructs `join` etc. and arbitrary user-defined functions (Java, Scala, Python). While powerful, implementing these invariants in terms of basics constructs like `join`, etc. is a very cumbersome manual process.

Instead, we took a closer look at the Flink CEP library. Complex Event Processing is a well-established approach for finding violations to invariants (see section 2.2.2).

Thus as a first step, we expressed some of the invariants with Flink CEP library using the Java programming language. The eShop’s Most Recent Price invariant I2 can be implemented as shown in listing 1.

Keep in mind this is only the pattern expressing the core logic of the invariant, the code making up the entire Flink job contains much more so-called boilerplate code. The full snippet is shown in appendix B.

We find this form of expressing invariants dissatisfying, especially compared to the natural language description of a relatively simple invariant.

Writing invariants directly in Java code has the benefit that it comes natural to developers, as they do not need to learn a new tool. However, for developers without prior experience with Apache Flink, or any other stream processing system for that matter, this approach might not be trivial.

Another downside is that writing invariants using Flink’s Java API, one loses the declarative aspect of the invariant and instead has to spell out exactly how the invariant should be checked. Thereby, focusing on **how** and not on **what** to check.

This brings us to our next section where we first describe the design process and finally, present our language for expressing invariants.

The impatient reader might already take a look at the outcome of our efforts in listing 3.

Extracting Common Properties

We are adopting the qualitative approach of coding [18] for extracting shared concepts from the mentioned invariants, which will then guide us when designing our invariant definition language.

When looking at the invariants, we identify the following key aspects:

Existence, Non-Existence of Events:

- “...*update event*...”
- “...*order completed event*...”
- “...*quote request event*...”

```

1 public static Pattern<EshopEvent, ?> InvariantPattern =
  ↳ Pattern.<EshopEvent>begin("firstPriceChange")
2   .subtype(ProductPriceChangedEvent.class)
3   .notFollowedBy("subsequentPriceChange")
4   .where(new IterativeCondition<>() {
5       @Override
6       public boolean filter(EshopEvent pc2, Context<EshopEvent> context)
7       ↳ throws Exception {
8           if (!pc2.EventName.equals("ProductPriceChangedIntegrationEvent"))
9               return false;
10          for (var pc1 : context.getEventsForPattern("firstPriceChange")) {
11              return pc1.EventBody.get("ProductId").asInt() ==
12                  ↳ pc2.EventBody.get("ProductId").asInt();
13          }
14      }
15      return false;
16  })
17  .followedByAny("userCheckoutWithOutDatedItemPrice")
18  .where(new IterativeCondition<>() {
19      @Override
20      public boolean filter(EshopEvent c, Context<EshopEvent> context)
21      ↳ throws Exception {
22          if (!c.EventName.equals("UserCheckoutAcceptedIntegrationEvent"))
23              return false;
24          for (var pc1 : context.getEventsForPattern("firstPriceChange")) {
25              var affectedProductId =
26                  ↳ pc1.EventBody.get("ProductId").asInt();
27              var newPrice = pc1.EventBody.get("NewPrice").asDouble();
28              for (Iterator<JsonNode> it =
29                  ↳ c.EventBody.get("Basket").get("Items").elements();
30                  it.hasNext();) {
31                  var item = it.next();
32                  if (affectedProductId == item.get("ProductId").asInt()
33                      && newPrice != item.get("UnitPrice").asDouble())
34                      return true;
35              }
36          }
37          return false;
38      }
39  })
40  .within(Time.seconds(30));

```

Listing 1: Most Recent Price invariant using Flink CEP API in Java

- “*..should be no updates...*”

Values of Events:

- “*...most recent price...*”
- “*...have a unique id...*”

Ordering of the events:

- “...followed by...”
- “After a...”

Time constraints:

- “...timely manner...”
- “...in reasonable time...”

State (materialized views) across Events:

- “...this stock should never be negative...”

We conclude that a language for expressing invariants about event-driven systems should have constructs for expressing these different concepts.

3.2 Invariant Language

Now we present our design of an invariant definition language; it draws strong inspiration from regular expressions, SQL, and CEP query languages (see section 2.2.2).

We ended up with 2 different forms of invariants that are only loosely connected; one is for expressing stateless invariants about event sequences and one is used for stateful invariants, where a state needs to be kept across multiple events.

The decision to separate the two is based firstly on our intent to keep the language simple. We believe mixing constructs for state and event sequences could result in unnecessarily complicated constructs, an example being `MATCH_RECOGNIZE` (see section 6.3). Secondly, this allows more freedom in terms of optimization and implementation when translating both types of invariants.

We acknowledge that having this separation might confuse developers. On the other hand, we also believe it might be beneficial to make it explicit when you reason about the state, which our choice forces the developer to do.

3.2.1 Stateless Invariants

Stateless invariants allow expressing properties about event existence, event values, their ordering, and time constraints. They are concerned with specifying a sequence, or pattern, of events and a condition representing the invariant

that has to be true if such a sequence is found. Additionally, a condition can be applied to a prefix match of the sequence (i.e. if only the beginning of the pattern has been spotted).

In figure 3.3 we present the syntax for stateless invariants, for the full grammar see appendix A.

```

eventName eventId [, ...]
  TOPIC: topicName
  SCHEMA: {attribute:type [, ...]}
[...]

SEQ eventSequence
[WITHIN duration {hour | min | sec | msec}]
[WHERE condition]
{
  ON FULL MATCH condition
| ON PREFIX MATCH prefix condition
  [...]
| ON FULL MATCH condition
  ON PREFIX MATCH prefix condition
  [...]
}

```

Figure 3.3: Syntax for stateless invariants.

The `eventName` can be considered the event type, for example `ProductBought` event.

The `eventId` is an alias for referencing events of that type later in the invariant, there can be multiple comma-separated eventIds for a single type.

The `topicName` is the log/topic name where the events are read from.

The specific event can have multiple `attribute`'s and each of these have a `type`, which can be one of `{number, string, boolean, timestamp}`.

The `eventSequence` specifies a pattern of `event` IDs, it is very similar to regular expressions. To stay concise we only present some example sequences and explain their semantics here, we go into more details in section 4.3.1.

- `(a, b)`: event `a` has to be followed by event `b`
- `(a, (b | c), d)`: event `a` can be followed by `b` or `c` event, but after either one there has to be a `d` event

- (a, b*, c): event a can be followed by zero or many b events until there is a c event
- (a, !b, c): between event a and event c there are no b events allowed

A **condition** contains predicates that consist of attributes of events (e.g. a.id), constants (e.g. 42, 'foo') and comparison operators (e.g. =, <, !=) between them. Multiple predicates can be combined with AND and OR.

The **prefix** after the **ON PREFIX MATCH** can be either ANY or a valid prefix of the original sequence.

3.2.2 Stateful Invariants

Next, we introduce our second form of invariants that allow reasoning about the state. Note, this type of invariant is not yet implemented in our tool and our contribution is limited to the design.

The fundamental idea is to declare some state in the form of a table, and then specify events that trigger an update to this state. Finally, one specifies an invariant that has to be true at all times, and will be checked after every state update. The proposed syntax is shown in figure 3.4. We contemplated introducing special constructs for defining the updates and the invariant, but instead decided on directly using SQL statements for both as this ensures sufficient expressiveness.

```

eventName eventId [, ...]
  TOPIC: topicName
  SCHEMA: {value:type [, ...]}
[...]

STATE state-definition

ON EVENT eventId
[WHERE condition]
EXECUTE sql-statement
[...]

INVARIANT sql-statement

```

Figure 3.4: Syntax for stateful invariants.

The definition of the events involved works the same as for stateless invariants. The **sql-statement** that is executed on a particular event can reference any of

the event’s attributes and has to be an `INSERT`, `UPDATE` or `DELETE` statement. The invariant `sql-statement` has access to the state in the form of the usual table abstraction and has to return a boolean, with `FALSE` indicating a violation of the invariant.

3.3 Expressibility

Our language for stateless invariants can be interpreted as a subset of the SASE+ language [19]. Our `ON FULL MATCH` condition can be added to the `WHERE` clause to a query. The behavior of our `ON PREFIX MATCH` can be emulated by instead writing multiple queries, one for each prefix with the same time limit in the `WITHIN` clause. One underlying evaluation model of SASE+ is NFA^b , a non-deterministic finite automaton with a match buffer [20]. The set of stream decision problems recognizable by NFA^b includes problems that are complete for nondeterministic space $\log n$ ($NSPACE(\log n)$). The idea for this is to show that a query can express directed graph reachability which is complete for $NSPACE(\log n)$.

The stateful invariants can construct arbitrary state using the values of events and inherit the full expressibility of SQL for formulating invariants about this state.

3.4 Invariants Revisited

We will now go through some of the invariants listed in section 3.1.1 and section 3.1.1 showing how to express them in our invariant language.

3.4.1 EshopOnContainers Invariants

Expressing eShop I1 - Overselling Items

For checking that no product is being oversold we have to consider `ProductCreated`, `ProductBought`, and `ProductStockChanged` events. We assume for now that `ProductDeleted` events are handled correctly, meaning there will be none of the three previously mentioned events after a product is deleted; this may also be checked with a different invariant. The resulting stateful invariant is shown in listing 2.

Expressing eShop I2 - Most Recent Price

This invariant expressed as a stateless invariant is shown in listing 3. To make sure we only consider the most recent `PriceChanged` event for a specific product, we specify the event sequence as `(pc1, !pc2, pb)`, where both `pc1` and `pc2` concern the same product. When now we first match an outdated

```

1 ProductCreatedEvent pc
2   TOPIC eshop_event_bus
3   SCHEMA {ProductId:number, Stock:number: Price:number,
4     ↪ Date:timestamp}
5 ProductBoughtEvent pb
6   TOPIC eshop_event_bus
7   SCHEMA {ProductId:number, Price:number, Date:timestamp}
8 ProductStockChangedEvent psc
9   TOPIC eshop_event_bus
10  SCHEMA {ProductId:number, NewStock:number, Date:timestamp}
11
12 STATE productStock (id INT, stock INT)
13
14 ON EVENT pc
15 EXECUTE (INSERT pc.id, pc.stock INTO productStock)
16
17 ON EVENT pb
18 EXECUTE (UPDATE productStock SET stock = stock - pb.amount
19   WHERE id = pb.id)
20
21 ON EVENT psc
22 EXECUTE (UPDATE productStock SET stock = psc.stock
23   WHERE id = psc.id)
24
25 INVARIANT (SELECT CASE WHEN EXISTS
26   (SELECT * FROM productStock WHERE stock < 0)
27   THEN FALSE
28   ELSE TRUE END);

```

Listing 2: eShop I1 Overselling Items Invariant

price change followed by a newer price change for the same product, the entire sequence is discarded and the invariant for the full match is not checked.

The **WHERE** clause specifies that all events relate to the same product ID. Finally, the full match condition is that the **ProductBought** event has the same price as the most recent price update event. The **WITHIN** clause specifies a 2-minute window, this is done for performance reason; we assume that 2 minutes should be sufficient for the system to update the price across all services, as we are dealing with an eventually consistent system.

```

1 ProductPriceChangedEvent pc1, pc2
2   TOPIC eshop_event_bus
3   SCHEMA {ProductId:number, NewPrice:number, Date:timestamp}
4
5 ProductBoughtEvent pb
6   TOPIC eshop_event_bus
7   SCHEMA {ProductId:number, Price:number, Date:timestamp}
8
9 SEQ (pc1, !pc2, pb)
10 WITHIN 2 min
11 WHERE (pc1.ProductId = pb.ProductId) AND
12        (pc1.ProductId = pc2.ProductId)
13 ON FULL MATCH (pc1.NewPrice = pb.Price)

```

Listing 3: eShop I2 Most Recent Price Invariant

Expressing eShop I3 - Timely Payment

In eShop, an `OrderSubmitted` event should be followed by a `PaymentSucceeded` or `PaymentFailed` event. We choose an arbitrary limit of 5 minutes, meaning if there is no payment event in that time frame after an `OrderSubmitted` event, we want to signal a violation. This time limit may be adjusted depending on how long actual payment processes take place. For finding violations we specify a prefix match with the condition `false`. This means every partial match will be considered a violation. The invariant is shown in listing 4.

Expressing eShop I4 - Deleting Items from Basket

Assume eShop would allow a user to delete their account, then the basket of said user should be deleted. This invariant is directly tied to avoiding accumulating inconsistent data across microservices and aims to monitor referential integrity. This invariant is easy to express using a stateless invariant, see listing 5. Again we choose an arbitrary limit of 5 minutes.

Expressing eShop I7 - Payment Amount Matches Product Prices

For this invariant (listing 6) we can make use of the “+” operator by matching all `ProductBought` events that are connected to a specific order or payment before matching a `PaymentSuccessful` event. In the full match condition, we specify that the sum of all `ProductBought` events connected to the order is the same as the total amount. Additionally, we can check that the entire process is completed by using a prefix match. Please note that the aggregate functions like `SUM` have yet to be implemented.

```

1 OrderSubmittedEvent os
2   TOPIC eshop_event_bus
3   SCHEMA {OrderId:number, Date:timestamp}
4
5 PaymentSucceededEvent ps
6   TOPIC eshop_event_bus
7   SCHEMA {OrderId:number, Date:timestamp}
8
9 PaymentFailedEvent pf
10  TOPIC eshop_event_bus
11  SCHEMA {OrderId:number, Date:timestamp}
12
13 SEQ (os, (ps | pf)) WITHIN 5 min
14 WHERE (os.OrderId = ps.OrderId OR os.OrderId = pf.OrderId)
15 ON PREFIX MATCH ANY false

```

Listing 4: eShop I3 Timely Payment Invariant

```

1 UserDeletedEvent ud
2   TOPIC eshop_event_bus
3   SCHEMA {UserId:number, Date:timestamp}
4 BasketDeletedEvent bd
5   TOPIC eshop_event_bus
6   SCHEMA {UserId:number, Date:timestamp}
7
8 SEQ (ud, bd) WITHIN 5 min
9 WHERE (ud.userid = bd.userid)
10 ON PREFIX MATCH ANY false

```

Listing 5: eShop I4 Deleting Items from Basket

```

1 PaymentSucceededEvent ps
2   TOPIC eshop_event_bus
3   SCHEMA {OrderId:number, TotalAmount:number, Date:timestamp}
4
5 ProductBoughtEvent pb
6   TOPIC eshop_event_bus
7   SCHEMA {OrderId:number, ProductId:number, Price:number,
8     ↪ Date:timestamp}
9
9 OrderPaymentSuccessful ops
10  TOPIC eshop_event_bus
11  SCHEMA {OrderId:number, Bill:number, Date:timestamp}
12
13 SEQ (ps, pb+, ops) WITHIN 10 min
14 WHERE (ps.OrderId = pb.OrderId AND ps.OrderId = ops.OrderId)
15 ON FULL MATCH (ps.TotalAmount = SUM(pb.Price)
16   AND ps.TotalAmount = ops.Bill)
17 ON PREFIX MATCH ANY false

```

Listing 6: eShop I7 Payment Amount Matches Product Prices

3.4.2 Lakeside Mutual Invariants

Expressing Lakeside I1 - Quote expiration

In Lakeside Mutual a customer can request an insurance quote from an insurance employee. After checking the customer's risk profile, the employee presents a policy offer with an expiration date to the customer. It is not allowed to have a `CustomerDecision` event after the quote has expired. We can express the invariant as shown in listing 7.

```

1 InsuranceQuoteExpiredEvent iqe
2   TOPIC customer-decision-topic
3   SCHEMA {QuoteId:number, Accepted:boolean, Date:timestamp}
4
5 CustomerDecisionEvent cd
6   TOPIC customer-decision-topic
7   SCHEMA {QuoteId:number, Accepted:boolean, Date:timestamp}
8
9 SEQ (iqe, !cd) WITHIN 2 min
10 WHERE expired.policyId = decisions.policyId
11 ON FULL MATCH false

```

Listing 7: Lakeside I1 Quote expiration Invariant

Expressing Lakeside I2 - Ghost Updates

After a policy is deleted there should be no further events concerning that policy, as that may indicate inconsistent data somewhere else in the system. The invariant in listing 8 checks that no event indicating a state change of the policy appears after deletion.

Expressing Lakeside I3 - Timely Creation

After a customer has accepted a quote a policy should be created. For this invariant (listing 9) we make use of both the full and partial match since whether or not a customer accepts is an attribute of the `CustomerDecision` event.

```

1 PolicyDeletedEvent pdeleted
2   TOPIC policy-created-topic
3   SCHEMA {PolicyId:number, Date:timestamp}
4
5 PolicyCreatedEvent pc
6   TOPIC policy-created-topic
7   SCHEMA {PolicyId:number, Date:timestamp}
8
9 PolicyUpdatedEvent pu
10  TOPIC policy-created-topic
11  SCHEMA {PolicyId:number, Date:timestamp}
12
13 PolicyDeniedEvent pdenied
14  TOPIC policy-created-topic
15  SCHEMA {PolicyId:number, Date:timestamp}
16
17 SEQ (pdeleted, (pc | pu | pdenied)) WITHIN 5 min
18 WHERE (pdeleted.PolicyId = pc.PolicyId
19        OR pdeleted.PolicyId = pu.policyId
20        OR pdeleted.PolicyId = pdenied.policyId)
21 ON FULL MATCH false

```

Listing 8: Lakeside I2 Ghost Updates Invariant

```

1 CustomerDecisionEvent ps
2   TOPIC customer-decision-topic
3   SCHEMA {QuoteId:number, Accepted:boolean, Date:timestamp}
4
5 PolicyCreatedEvent ps
6   TOPIC policy-created-topic
7   SCHEMA {QuoteId:number, Date:timestamp}
8
9 SEQ (cd, pc) WITHIN 1 min
10 WHERE (cd.QuoteId = pc.QuoteId)
11 ON FULL MATCH cd.Accepted = true
12 ON PREFIX MATCH cd.Accepted = false

```

Listing 9: Lakeside I3 Timely Creation Invariant

Chapter 4

Monitoring Event-Driven Microservices

4.1 Overall Architecture

In this chapter, we will describe the overall architecture of our tool¹, and discuss the implementation details of its components. The architecture of the tool while interacting with an event-driven architecture is shown in figure 4.1.

The key components are:

- the **Workload Generator**, which executes a given scenario as requests against the application
- the **Invariant Translator**, which translates invariants expressed in the DSL described in section 3.2 into Flink jobs, which can be submitted to a Flink cluster, allowing real-time monitoring of the invariants

We will now describe the functionality of each component in detail and discuss our design decisions and implementation details along the way.

4.2 Workload Generator

To monitor any safety properties of an application a necessary first step is to execute a workload against the system. This workload should precisely trigger the behavior of the system we are interested in monitoring. Our goal is to effectively model realistic users of the system. We aim at generating a qualitative load that triggers events in a way that is relevant to the monitored invariants. We want to model the behavior of users that might lead to violations of the

¹<https://github.com/otter-diku/simulation-framework-for-microservices>

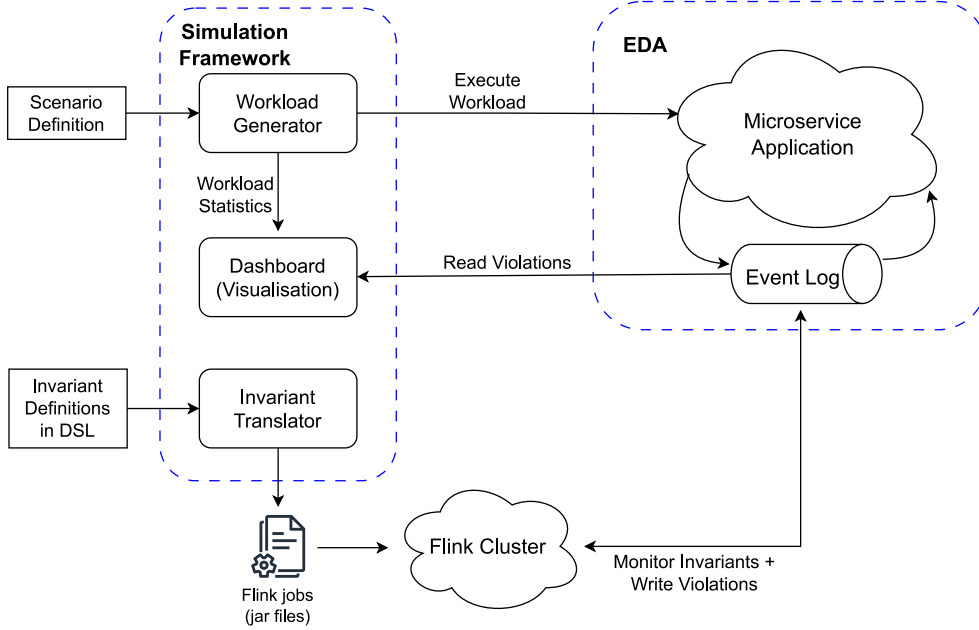


Figure 4.1: Architecture of the Simulation Tool

invariants. For this, it is important to control the concurrency of the workload and the timing of specific operations.

While some tools^{2,3} exist for load testing, we decided to create our own workload generation component. This allows us to have full control over the workload execution and its definition. Instead of writing executable code, we can instead describe the workload in precise terms like transactions and relevant run-time parameters. This is also the form in which benchmarks like YCSB [21] are presented.

4.2.1 Workload Definition

After exploring different ways to model workloads we decided a specific workload should consist of one or many transactions. A single transaction corresponds to a specific user behavior and can consist of an arbitrary number of operations. We chose JSON as the format to define these different parts of a workload. This choice is arbitrary and any other configuration language could have been used.

Next, we discuss the different parts of a workload definition. We will only describe the fundamental pieces, a full treatment of the configuration language would go beyond the scope of this thesis and potentially bore the reader.

²<https://k6.io/>

³<https://locust.io/>

- **Operations:** Operations are the fundamental unit of work executed by the workload generator. We assume that monitored applications expose an HTTP API, therefore the types of operations currently supported are HTTP(s) requests (see figure 4.2) and sleep (see figure 4.3).

Further, an operation has access to arguments that it may receive from previous operations of the same transaction, from the transaction configuration or workload configuration. An operation can expose return values for subsequent operations, these are either parts of the JSON response payload or the response header.

```
{
  "templateId": "op_do_work",
  "type": "http",
  "httpMethod": "post",
  "arguments": [
    {
      "name": "value",
      "type": "number"
    }
  ],
  "requestPayload": {
    "type": "json",
    "content": {
      "number": "{{value}}"
    }
  },
  "url": "api/endpoint"
}
```

Figure 4.2: Example operation sending a JSON payload to an HTTP endpoint.

- **Transactions:** Transactions model a single user action that gets executed against the application, and in this sense adhere to the usual meaning of transactions in the context of database systems [22]. A transaction consists of a sequence of multiple operations. These operations are executed one after another and the potential return values of an operation are passed to the next one in the sequence. In figure 4.4 the configuration of an example transaction is shown, it consists of the previously defined operations.
- **Workloads:** The workload configuration file contains transactions and key run-time parameters of the workload to be executed.

```

{
  "templateId": "op_sleep_in_sec",
  "arguments": [
    {
      "name": "sleep-time",
      "type": "number"
    }
  ],
  "type": "sleep",
  "duration": "{{sleep-time}}",
  "units": "seconds"
}

```

Figure 4.3: Example sleep operation

```

{
  "templateId": "tx_sleep_work_sleep",
  "operations": [
    {
      "operationReferenceId": "op_sleep_in_sec",
      "id": "sleep-1"
    },
    {
      "operationReferenceId": "op_do_work",
      "id": "op-1"
    },
    {
      "operationReferenceId": "op_sleep_in_sec",
      "id": "sleep-2"
    }
  ]
}

```

Figure 4.4: Example transactions that sleeps and does some work.

The “**maxConcurrentTransactions**” value determines the maximum number of concurrent users interacting with the application. To generate data for the transactions, the user can specify multiple “**generators**”. A generator may produce values of a certain type like numbers and strings according to a distribution (constant, uniform, Zipfian). Finally, these generators are referenced when listing the transactions contained in the

workload.

```
{
  "templateId": "example_workload",
  "maxConcurrentTransactions": 15,
  "generators": [
    {
      "id": "example-generator",
      "type": "unsignedInt",
      "distribution": "uniform",
      "min": 1,
      "max": 50
    }
  ],
  "transactions": [
    {
      "id": "transaction-1",
      "transactionReferenceId": "tx_sleep_work_sleep",
      "count": 1000,
      "data": [
        {
          "name": "value",
          "generatorReferenceId": "example-generator"
        }
      ]
    }
  ]
}
```

Figure 4.5: Example workload configuration

4.2.2 Design and Implementation

The challenging aspect of the workload generator is making it scale to a large number of concurrent and potentially long-running transactions. In our design, we utilized the Microsoft Orleans [23] framework, as for our setting a single user corresponds nicely to the concept of a virtual actor (grain) in the Orleans framework.

Thus, the maximum number of concurrent transactions is the number of actors initialized when starting the workload generation. In turn, our workload scheduler component sends the transactions that are to be executed to individual actors (grains) through a queue.

4.3 Invariant Translator

Next, we explain how the language for defining invariants, described in section 3.2, is translated into Java code that can be compiled into Apache Flink jobs. Note that currently, we have only fully implemented stateless invariants, thus the following discussion solely applies to them.

4.3.1 Design and Implementation

We somewhat stumbled into creating the invariant definition language midway into the thesis. Therefore, we think our implementation of the translation process lacks some clarity and may be somewhat unorthodox.

We structured the translation process into the following steps:

1. Parse the invariant into a parse-tree representation
2. Build an internal representation from the parse-tree
3. Analyze and optimize the internal representation
4. Generate Java code

For step one we used the excellent ANTLR⁴ parser generator, the full grammar can be found in appendix A.

In the second step, we validate and build an internal representation of the invariant which is centered around the event sequence. Besides checking the basic correctness of the invariant, e.g. only event IDs are used that are also listed beforehand, we have to enforce certain limitations about which invariants can be expressed. This arises naturally from the constructs involved or from the limitations of the underlying implementation of Flink CEP.

For the event sequence **SEQ** we disallow:

- wild cards combined with negated events **!a***, **!a+**
- duplicate event IDs (**a**, **b**, **a**); events with the same type need unique event IDs, i.e. (**a1**, **b**, **a2**)
- events combined by **or** (**|**) to have same type: (**a1** **|** **a2**)
- negated events to be combined with **or**: (**a** **|** **!b**)
- the event sequence to start with negated events: (**!a** , **b**)

⁴<https://wwwantlr.org/>

- a negated event be preceded by an optional one: `(a*, b!)`
- nested `or (|)` combinations of events: `(a | (b | c))`
- event sequences that end with a negated event missing a `WITHIN` clause `(a, !b) WHERE ...`

For the `WHERE` clause, we require it to be of the following form: `term0 AND term1 AND ...`, that is a conjunction of multiple terms, where each term may consist of arbitrary combinations of equalities and logical connectives.

Each term must adhere to the following limitation: a term that references a negated event or an event used in an `or (|)` clause cannot reference events that occur later than this event in a sequence.

That is, given a sequence `SEQ (a, !b, (c|d), e)`:

- a term `b.id = e.id` is not allowed
- a term `c.id = e.id` is not allowed

The last step of generating Java code caused us some trouble.

Our first approach consisted of translating the invariants into Java functions; these functions would later be passed to Flink API and executed. This approach turned out to be fruitless as we ran into problems with serialization. As Flink is a distributed framework, the programs get serialized so that each worker node can receive code to be executed along with the required context.

Searching for a solution we then took a closer look at how the Flink team implemented⁵ the `MATCH_RECOGNIZE` SQL construct on top of Flink CEP (we further discuss match recognize in section 6.3). To support `MATCH_RECOGNIZE` Flink generates Java code instead of constructing functions in memory. This approach can be cumbersome to implement and maintain, but we were confident that it would work.

Thus, on the second try, we generate Java code from our internal representation of the invariant. The resulting Java files are then compiled with maven into `.jar` files that can be submitted to a Flink cluster.

The basic idea for generating the invariant code is to map the event sequence and `WHERE` clause directly to a Flink CEP pattern. The terms in the `WHERE` clause have to be analyzed and placed as iterative conditions between the patterns depending on whether they involve

⁵<https://github.com/apache/flink/pull/6815>

The invariant, i.e. the `ON FULL MATCH` condition is implemented in the process match function provided by the Flink CEP library. The `PREFIX MATCH` conditions are implemented in the process time-out match function with logic for distinguishing the different prefixes from one another.

This means we utilize the Flink CEP API to create the code for matching relevant patterns that are subject to the invariant. The logic for checking the invariant itself is then implemented as a user-defined function.

Our implementation can only be seen as a proof of concept and the translation of invariants needs a more rigorous treatment.

Optimizations

As the user specifies the invariants in our declarative language there is the possibility to optimize the internal representation and generate efficient code for the Flink run time.

Query optimization is a broad subject with a lot of depth [24, 25, 26]. It seems worthwhile to investigate different methods that apply to our invariant language, especially since we are targeting a stream processing framework.

In this thesis, we only present a very basic optimization. The optimization is to automatically detect when we can use a keyed stream, meaning when we can partition events based on a key. Using a keyed stream allows for better parallelization as each partition can be processed by an independent Flink operator.

The simple observation that makes this optimization possible is that there needs to be a transitive chain of equalities between all events of the sequence in the `WHERE` clause. The key selector function for Flink then simply extracts the attribute for each.

If we consider the `WHERE` clause of the Timely Payment invariant (see listing 4)

```
WHERE (os.OrderId = ps.OrderId OR os.OrderId = pf.OrderId)
```

we see that the key for all elements is the `OrderId` attribute.

Thus, we would get the following key selector function:

```
var keyedStream = filteredStream.keyBy((e) ->{
    if (e.EventName.equals("OrderSubmittedEvent")) {
        return e.EventBody.get("OrderId").asInt();
    } else if (e.EventName.equals("PaymentSucceededEvent")) {
        return e.EventBody.get("OrderId").asInt();
    } else if (e.EventName.equals("PaymentFailedEvent")) {
        return e.EventBody.get("OrderId").asInt();
    }
    throw new RuntimeException("Unexpected Event Type");
});
```

Vision for Translating Stateful Invariants

We did not manage to implement the stateful invariants proposed in section 3.2.2.

Before deciding to write our DSL we implemented the eShop I1 - Overselling Items invariant directly in Java on top of the Flink Datastream API using keyed state (see appendix C). We thought about basing the stateful invariants on this approach, but decided against it, because had we used keyed state it's not obvious what the expressibility of our language to create and query state would be.

As an alternative for implementing our stateful invariants, we looked into Flink SQL which allows to develop streaming applications using standard SQL. Flink SQL is centered around continuous queries and dynamic tables which continuously produce a result. Flink SQL does not allow for `UPDATE` or `DELETE` statements which we would need for our implementation of stateful invariants.

Therefore, our current vision is to use an embedded SQL database like DuckDB⁶ to model the state for the invariant as a table - each event may trigger updates to that table, and the global invariant checks a certain SQL statement after each update.

⁶<https://duckdb.org/>

Chapter 5

Demonstration

5.1 Example Application: eShopOnContainers

A natural question to ask at this point is: *How can a developer make use of this tool?*. For that, we will now describe an example use case of our simulation framework applied to the eShopOnContainers application [1].

We continue the story from the motivation (section 1.1) and take the perspective of a software engineer working on an online store.

The developers know there is no silver bullet in software engineering [27], which is also true for microservices. They are convinced the advantages they care about will manifest themselves but the potential downsides have to be considered.

A primary goal is to ensure the scalability of the online-store to handle traffic spikes. The team knows that enforcing ACID guarantees in a distributed setting with two-phase commit is expensive, instead they consciously adopt eventual consistency [28] by using an event-driven architecture.

They are aware that this may lead to inconsistent state across the different data stores of the microservices and cause problems, as the consistency of certain shared state is business critical. For example, a product's price inside a customer's basket is only updated through asynchronous `PriceChanged` events emitted by the catalog service.

Given this situation, the team might find itself wondering about:

1. How often do the inconsistencies occur given typical website traffic? Can they be ignored or does the consistency between microservices need to be improved (using, for example, the Saga pattern)

2. How well does the current solution scale? The issue might not be significant at the moment, but what happens if the eShop starts attracting more and more traffic?

The development team is therefore interested in a solution that would allow them to measure and benchmark the degree of data consistency problems in their solution.

They first need to have a way of simulating production-like traffic to their website. Second, they want to be able to easily express some ground truths (or invariants) about their system, that they hope would hold, even under an increased workload. Any violations of these invariants should be easy to monitor.

Luckily, one of the developers seems to have found the perfect tool for the job, namely Otter-Diku's simulation framework for microservices.

5.2 Preliminary assessment

We assume that the eShop's development team is interested in benchmarking the Most Recent Price Change (section 3.4.1) and the Timely Payment (section 3.4.1) invariants.

They suspect (figure 5.1) that there might be some data consistency issues if users are buying products from the website, while the catalog product prices are being updated. Also, the payment processing may take too long.

5.3 Workload

The team starts defining a load test that should run against the eShop. The load test will consist of a workload specification (listing 10), containing one or more transactions (listing 11). Each transaction will then contain one or more operations (listing 12) - for example, to buy a product, a user might need to get it from the catalog, add it to the basket and finish by checking out said basket.

They choose to spawn 100 concurrent virtual users, each of them either updating an item's price or purchasing a product. They believe this should stress test the system enough to discover potential downfalls of their implementation.

5.4 Specifying the invariants

The team then proceeds to express their invariant using our custom DSL (listing 3 and listing 4). We assume Apache Kafka is used as the event-distribution

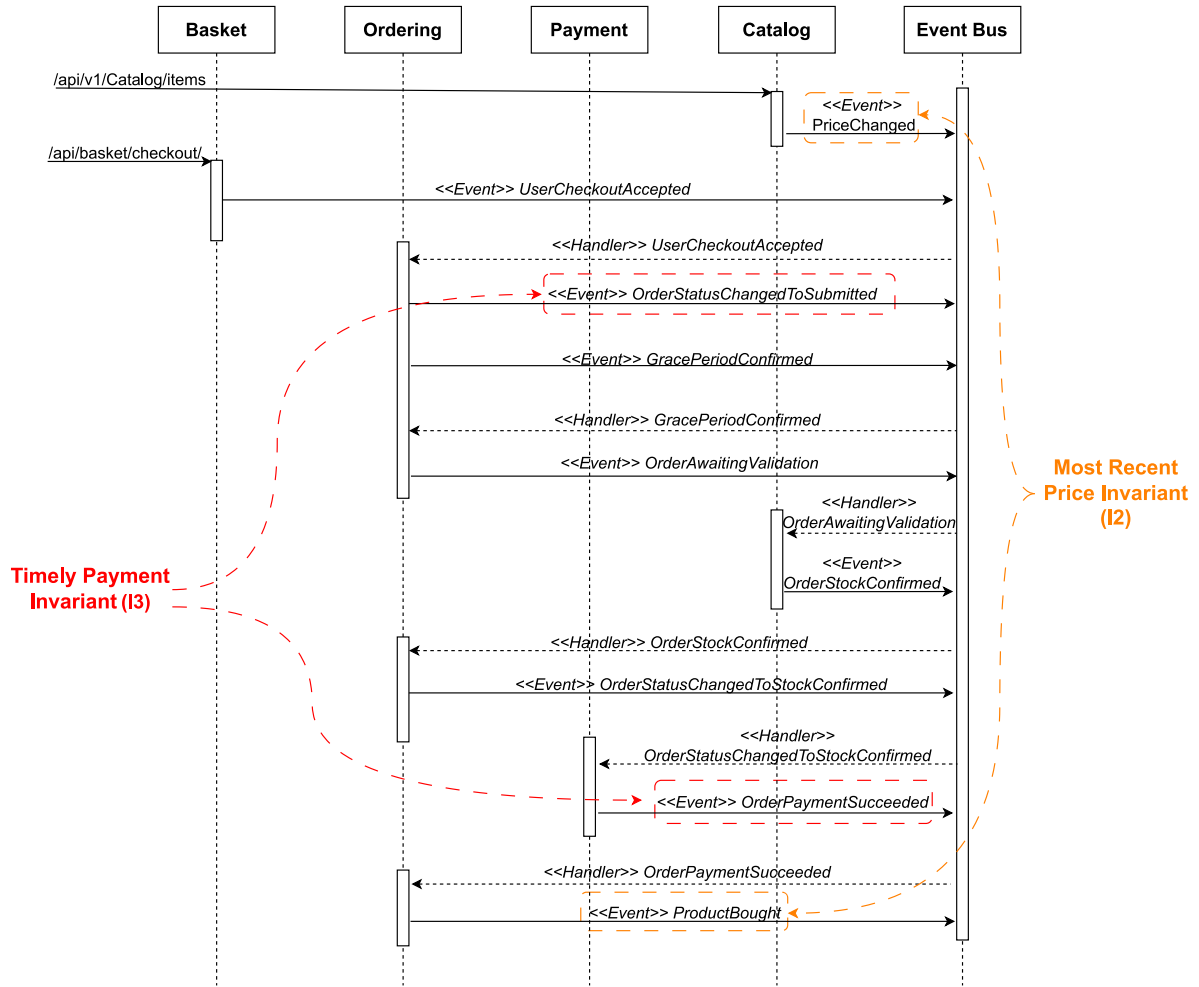


Figure 5.1: eShop Events as sequence diagram

layer. Therefore, they need to create a simple configuration file for Kafka (listing 13); this file contains necessary credentials and the Kafka topic to which the violations should be sent.

With these files in place, the Invariant Translator generates the `.jar` files that get uploaded to a Flink cluster - either running locally or hosted.

5.5 Results

The setup is now done and the team can start monitoring the behavior of their system.

In general, it is up to the user to choose how they want the data from the Workload Generator and the Flink cluster visualized. In our example, we use

Seq [29] to conveniently present data in one dashboard:

- logs from the work being executed by the Workload Generator
- events being published in the system under test (here: eShop)
- observed violations of the specified invariants

This should be a good base for the team to start tweaking their solution - they now have a reproducible way of benchmarking their application.

Should they decide to improve their data consistency issues by implementing different architectural patterns or locking mechanisms, our tool can be used for measuring the impact of such changes.



Figure 5.2: Example dashboard displaying transactions executed and violations detected

```
{
  "maxConcurrentTransactions": 100,
  "generators": [
    {
      "id": "productId",
      "type": "unsignedInt",
      "distribution": "uniform",
      "min": 1,
      "max": 20
    }
    // [...]
  ],
  "transactions": [
    {
      "transactionReferenceId": "tx_update_catalog_price",
      // [...]
      "count": 1000,
      "data": [
        {
          "name": "product-id",
          "generatorReferenceId": "productId"
        }
      ]
    },
    {
      "transactionReferenceId": "tx_buy_single_item",
      // [...]
      "count": 1000,
      "data": [
        {
          "name": "product-id",
          "generatorReferenceId": "productId"
        }
      ]
    }
  ]
}
```

Listing 10: Example workload specification


```
{
  "templateId": "tx_buy_single_item",
  "operations": [
    {
      "operationReferenceId": "op_get_catalog_item_by_id",
      "id": "op-1"
    },
    {
      "operationReferenceId": "op_basket_add_single_item",
      "id": "op-2"
    },
    {
      "operationReferenceId": "op_basket_checkout",
      "id": "op-3"
    }
  ]
}
```

Listing 11: Example transaction specification

```
{
  "templateId": "op_get_catalog_item_by_id",
  "type": "http",
  "httpMethod": "get",
  "arguments": [
    {
      "name": "product-id",
      "type": "number"
    }
  ],
  "response": {
    "payload": {
      "type": "json",
      "returnValues": [
        {
          "key": "item-1",
          "value": "$",
          "type": "object"
        }
      ]
    }
  },
  "url":
    ↪ "https://example.com/api/v1/Catalog/items/{{product-id}}"
}
```

Listing 12: Example operation specification

```
{
  "broker": "<kafka-broker-url>",
  "username": "<kafka-sasl-username>",
  "password": "<kafka-sasl-password>",
  "violationsTopicName": "eshop-violations" // the Kafka topic
    ↪ where the violations should be written to
}
```

Listing 13: Example operation specification

Chapter 6

Related Work

6.1 HawkEDA

The HawkEDA [30] project is the predecessor to our tool, it is based on the same fundamental idea. The goal of HawkEDA is to quantify data integrity violations in event-driven systems. Our thesis directly builds on top of this work by adding a DSL for invariant definition and creating a functional tool that can be applied to different microservice applications.

6.2 Preventing Violations

In contrast to our monitoring approach there exist several approaches that directly aim to prevent violations of invariants from happening in the first place. For example, orchestration using the Saga pattern is a common approach to coordinate cross-microservice transactions to achieve transactional guarantees, such as ACID properties or a subset thereof [31]. Further, the concept of a centralized coordination service that controls the flow of events between microservices has been proposed in [32]. An approach to codifying and enforcing invariants at the event distribution is demonstrated in [33]

Using such frameworks and patterns guarantee that invariants or parts of them are not violated. This is more powerful than pure monitoring but also implies that the team of developers has to design the application with the framework in mind. This imposes new restrictions that potentially all microservices have to adhere to. These global constraints of the framework go against the principle that in microservice applications multiple teams of developers should be able to employ different technologies and evolve their services independently.

6.3 SQL Match Recognize Construct

In December 2016, the International Organization for Standardization (ISO) released a new version of the SQL standard which includes Row Pattern Recognition [34]. The `MATCH_RECOGNIZE` construct accepts a set of rows (from a table, view, sub-query, or other source) as input, and returns all matches for a given row pattern within this set. The pattern is defined similarly to a regular expression. The clause can return either all the rows belonging to each match or one summary row per match.

The `MATCH_RECOGNIZE` construct is typically used to detect events in time series. Flink SQL also supports this construct, making it a potential candidate for expressing our invariants.

Thus, we tried to express the eShop I2 - Most Recent Price and eShop I3 - Timely Payment invariant using this approach and failed. As the SQL standard for `MATCH_RECOGNIZE` does not support a `WITHIN` clause it is not obvious how to implement the Timely Payment invariant.

Interestingly, the Flink implementation of `MATCH_RECOGNIZE` supports a `WITHIN` clause after the pattern, but this is mostly for performance reasons, allowing pruning state after the time window closes. But for the invariant, we instead want to trigger a violation if a pattern has not been completed inside the time window, which does not seem to be possible.

For illustration, we show the typical example (see listing 14) that is used to introduce the `MATCH_RECOGNIZE`; it's a query to find periods of a constantly decreasing price of a stock.

Generally, the `MATCH_RECOGNIZE` clause can only be used to find patterns, meaning we cannot declare invariants and instead need to declare patterns that would violate the invariants.

Considering its verbosity and no direct way to reason about the absence of events, we conclude that the `MATCH_RECOGNIZE` clause is not well suited to express the invariants we are targeting with our DSL.

6.4 Runtime Monitoring

Another field closely connected is the discipline of runtime monitoring, here properties and invariants to monitor are typically expressed in formal languages, often some form of temporal logic.

The Monpoly tool [35] allows checking formulas expressed in metric first-order temporal logic (MFOTL) with aggregation operators [36]. We show how the eShop I2 - Most Recent Price invariant can be expressed using MFOTL.

```

1  SELECT *
2  FROM Ticker
3      MATCH_RECOGNIZE (
4      PARTITION BY symbol
5      ORDER BY rowtime
6      MEASURES
7          START_ROW.rowtime AS start_tstamp,
8          LAST(PRICE_DOWN.rowtime) AS bottom_tstamp,
9          LAST(PRICE_UP.rowtime) AS end_tstamp
10     ONE ROW PER MATCH
11     AFTER MATCH SKIP TO LAST PRICE_UP
12     PATTERN (START_ROW PRICE_DOWN+ PRICE_UP)
13     DEFINE
14         PRICE_DOWN AS
15             (LAST(PRICE_DOWN.price, 1)
16              IS NULL AND PRICE_DOWN.price < START_ROW.price) OR
17              PRICE_DOWN.price < LAST(PRICE_DOWN.price, 1),
18         PRICE_UP AS
19             PRICE_UP.price > LAST(PRICE_DOWN.price, 1)
20 ) MR;

```

Listing 14: Query using MATCH_RECOGNIZE to find periods of a constantly decreasing price of a single ticker

First, one has to define the signature of the events involved in the property. In our case, we have:

```

price_update(i_id:int, price:float)
checkout_item(i_id:int, price:float)

```

The invariant can then be expressed using the following formula

```

checkout_item(i_id, price) AND
(NOT EXISTS p. price_update(i_id, p) SINCE[0,30]
 price_update(i_id, p)) AND
NOT (price = p)

```

Then, given a following log of events

```
@5 checkout_item (2,200)
@10 price_update (1,100)
@20 checkout_item (2,200)
@25 checkout_item (1,200)
```

the monitor detects a violation:

```
@25 (time point 3): (1,200,100)
```

Although the formulas have a precise semantically defined meaning, one can make the case that this form of expressing invariants is not very accessible to a regular developer and requires expert knowledge. Generally, this approach to runtime monitoring has concrete theoretical foundations with superior correctness guarantees, making it well-suited for high-risk scenarios.

Chapter 7

Conclusion

Overall, this thesis demonstrates that a tool like ours can help developers to deal with real problems that arise when building event-driven microservice applications. Although our implementation is far from complete it proves the feasibility of the approach. As the current state-of-the-art for developing such applications in the industry seems unsatisfactory, pushing further in this direction seems worthwhile.

The decision of writing our workload generator turned out to be sub-optimal as it took a substantial amount of time that could have been used to refine the invariant DSL, without contributing much compared to existing solutions on the market [37, 38].

While designing the invariant language we were influenced by Apache Flink CEP being the underlying technology. In hindsight, this turned out to be too constraining and led us to create a language very close to SASE+. However, considering we somewhat stumbled into the process of designing the language, we are still happy with the outcome and believe it can be useful.

The purpose of our DSL is to ease the use of stream processing and CEP, making these technologies more accessible. We hope that given a list of requirements regarding the data integrity in the system, a developer can quickly integrate our tool with their application and get an insight into possible data consistency violations in their system. This knowledge should help them and their team to further tweak and evolve their solution.

Although for observing some invariants a tool like ours is not a necessity, we believe that it provides a compelling and easy-to-use approach.

7.1 Future Work

More work can be done to further extend the expressibility of the language, like adding stateful invariants using SQL syntax for updating the state on events.

Also, including quantifiers (`for all`, `exists`) and aggregate functions (`sum`, `avg`, etc.) to `WHERE` clauses and invariant checks would be beneficial.

It would also be possible to extend the scope of applications that could take advantage of our tool. During our research, we investigated a setup of Change Data Capture (CDC) for the SocialNetwork microservice application from the DeathStarBench suite [39]. Though this work is not included in this thesis, we believe that utilizing CDC (e.g. `debezium`[40]) would allow us to monitor relevant invariants for applications that don't follow the event-driven architecture.

Bibliography

- [1] Net application architecture reference apps. *eShopOnContainers*. <https://github.com/dotnet-architecture/eShopOnContainers>. Accessed: 2023-02-08.
- [2] Sam Newman. *Building Microservices, 2nd Edition*.
- [3] IBM Team. Microservices in the enterprise, 2021: Real benefits, worth the challenges. Technical report, International Business Machines Corporation, 2021.
- [4] Scaling up the prime video audio/video monitoring service and reducing costs by 90 <https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90>. Accessed: 2023-05-30.
- [5] Rodrigo Laigner, Yongluan Zhou, Marcos Antonio Vaz Salles, Yijian Liu, and Marcos Kalinowski. Data management in microservices: state of the practice, challenges, and research directions. *Proceedings of the VLDB Endowment*, 14(13):3348–3361, 2021.
- [6] Muhammad et al. Waseem. Understanding the issues, their causes and solutions in microservices systems: An empirical study. *IEEE Transactions on Software Engineering*, 2023.
- [7] Service weaver framework by google introduced in march 2023. <https://opensource.googleblog.com/2023/03/introducing-service-weaver-framework-for-writing-distributed-applications.html>. Accessed: 2023-05-30.
- [8] Solace 2021 survey results - the great eda migration. <https://solace.com/resources/white-papers/wp-download-the-great-eda-migration-2021-survey-results#main-content>. Accessed: 2023-03-30.
- [9] Martin et al. Kleppmann. Online event processing: Achieving consistency where distributed transactions have failed. *ACM Queue*, 2019.

- [10] Marco et al. Bucci. Foundations of complex event processing. *arXiv preprint:1709.05369v2*, 2018.
- [11] Sebastian Meinecke. Real-time monitoring of clinical processes using complex event processing and transition systems. *European Federation for Medical Informatics and IOS Press*, 2014.
- [12] Charles Antony Richard Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [13] Gerwin et al. Klein. sel4: Formal verification of an os kernel. In *ACM Symposium on Operating Systems Principles*, pages 207–220, 2009.
- [14] Xavier Leroy, Sandrine Blazy, Daniel Kästner, Bernhard Schommer, Markus Pister, and Christian Ferdinand. CompCert-a formally verified optimizing compiler. In *ERTS 2016: Embedded Real Time Software and Systems, 8th European Congress*, 2016.
- [15] Microservice-api-patterns. *Lakeside Mutual*. <https://github.com/Microservice-API-Patterns/LakesideMutual>. Accessed: 2023-02-08.
- [16] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, et al. The dataflow model: a practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing. 2015.
- [17] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4), 2015.
- [18] Carl Auerbach and Louise B Silverstein. *Qualitative data: An introduction to coding and analysis*, volume 21. NYU press, 2003.
- [19] Daniel Gyllstrom, Jagrati Agrawal, Yanlei Diao, and Neil Immerman. On supporting kleene closure over event streams. In *2008 IEEE 24th International Conference on Data Engineering*, pages 1391–1393. IEEE, 2008.
- [20] Jagrati Agrawal, Yanlei Diao, Daniel Gyllstrom, and Neil Immerman. Efficient pattern matching over event streams. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 147–160, 2008.
- [21] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with ycsb. In

- Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154, 2010.
- [22] Philip A Bernstein and Eric Newcomer. *Principles of transaction processing*. Morgan Kaufmann, 2009.
- [23] Phil Bernstein, Sergey Bykov, Alan Geller, Gabriel Kliot, and Jorgen Thelin. Orleans: Distributed virtual actors for programmability and scalability. *MSRTR2014*, 41, 2014.
- [24] Goetz Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys (CSUR)*, 25(2):73–169, 1993.
- [25] Luping Ding, Songting Chen, Elke A Rundensteiner, Junichi Tatemura, Wang-Pin Hsiung, and K Selcuk Candan. Runtime semantic query optimization for event stream processing. In *2008 IEEE 24th International Conference on Data Engineering*, pages 676–685. IEEE, 2008.
- [26] Haopeng Zhang, Yanlei Diao, and Neil Immerman. On complexity and optimization of expensive queries in complex event processing. In *SIGMOD*, pages 217–228, 2014.
- [27] Frederick Brooks and H Kugler. *No silver bullet*. April, 1987.
- [28] Sebastian Burckhardt et al. Principles of eventual consistency. *Foundations and Trends® in Programming Languages*, 1(1-2):1–150, 2014.
- [29] Seq. <https://datalust.co/seq>. Accessed: 2023-05-29.
- [30] Prangshuman Das, Rodrigo Laigner, and Yongluan Zhou. Hawkeda: a tool for quantifying data integrity violations in event-driven microservices. In *Proceedings of the 15th ACM International Conference on Distributed and Event-based Systems*, pages 176–179, 2021.
- [31] Martin Štefanko, Ondřej Chaloupka, Bruno Rossi, et al. The saga pattern in a reactive microservices environment. In *Proc. 14th Int. Conf. Softw. Technologies (ICSOFT 2019)*, pages 483–490. SciTePress Prague, Czech Republic, 2019.
- [32] Tilman Zuckmantel, Yongluan Zhou, Boris Döder, and Thomas Hildebrandt. Event-based data-centric semantics for consistent data management in microservices. In *Proceedings of the 16th ACM International Conference on Distributed and Event-Based Systems*, pages 97–102, 2022.
- [33] Anna Lesniak, Rodrigo Laigner, and Yongluan Zhou. Enforcing consistency in microservice architectures through event-based constraints. In *Proceedings of the 15th ACM International Conference on Distributed and Event-based Systems*, pages 180–183, 2021.

- [34] Jan Michels, Keith Hare, Krishna Kulkarni, Calisto Zuzarte, Zhen Hua Liu, Beda Hammerschmidt, and Fred Zemke. The new and improved sql: 2016 standard. *ACM SIGMOD Record*, 47(2):51–60, 2018.
- [35] David A Basin, Felix Klaedtke, and Eugen Zălinescu. The monpoly monitoring tool. *RV-CuBES*, 3:19–28, 2017.
- [36] David Basin, Felix Klaedtke, Srdjan Marinovic, and Eugen Zălinescu. Monitoring of temporal first-order properties with aggregations. *Formal methods in system design*, 46:262–285, 2015.
- [37] Locust - an open source load testing tool. <https://locust.io>. Accessed: 2023-03-30.
- [38] k6s - an open source load testing tool from grafana labs. <https://k6s.io>. Accessed: 2023-03-30.
- [39] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 3–18, 2019.
- [40] debezium - open source distributed platform for change data capture. <https://debezium.io>. Accessed: 2023-03-30.

Appendix A

Invariant Language Grammar

```
1 grammar Invariants;
2
3 invariant: eventDefinition* query EOF;
4
5 query
6   : 'SEQ' '(' events ')' '\n'?
7     ('WITHIN' time)? '\n'?
8     ('WHERE' where_clause)? '\n'?
9     on_full_match '\n'?
10    (on_prefix_match '\n'?)*
11  | 'SEQ' '(' events ')' '\n'?
12    ('WITHIN' time)? '\n'?
13    ('WHERE' where_clause)? '\n'?
14    (on_prefix_match '\n'?)+
15    ;
16
17 on_full_match: 'ON FULL MATCH' invariant_clause;
18 on_prefix_match: 'ON PREFIX MATCH' prefix invariant_clause;
19
20 prefix
21   : default_prefix
22   | '(' events ')';
23
24 default_prefix: 'DEFAULT';
25
26 invariant_clause
27   : lpar term rpar (and lpar term rpar)*
28   | BOOL
29   ;
30
31 eventDefinition: eventType eventId
32                 'topic:' topic
33                 'schema:' schema '\n'?
34                 ;
35 eventType: IDENTIFIER;
36 eventId: IDENTIFIER;
```

```

37  topic: IDENTIFIER;
38  schema: '{' schemaMember (',' schemaMember)* '}';
39
40  schemaMember: IDENTIFIER ':' memberType;
41
42  memberType: 'number' | 'string' | 'bool' | 'timestamp';
43
44  events
45    : event (',' event)*
46    ;
47
48  event
49    : eventAtom
50    | orOperator
51    ;
52
53  orOperator: '(' eventId ('|' eventId)* ')' regexOp?;
54
55  regexOp
56    : '*'
57    | '+'
58    ;
59
60  negEvent: '!' eventId;
61
62  eventAtom
63    : negEvent
64    | eventId regexOp?;
65
66
67  where_clause
68    : lpar term rpar (and lpar term rpar)*
69    ;
70
71  term
72    : equality
73    | term and term
74    | term or term
75    | lpar term rpar
76    ;
77
78  lpar: '(';
79  rpar: ')';
80
81  equality
82    : quantity EQ_OP quantity;
83
84  quantity
85    : atom
86    | qualifiedName
87    ;
88
89  atom

```

```

90     : BOOL
91     | INT
92     | STRING
93     ;
94
95 qualifiedName
96     : IDENTIFIER ('.' IDENTIFIER)*
97     ;
98
99 time: INT TIME;
100
101 and: 'AND';
102 or: 'OR';
103
104 EQ_OP: '=' | '!=' | '<' | '<=' | '>' | '>=';
105 INT: '-'?[0-9]+;
106 STRING: '\"' IDENTIFIER '\"';
107 TIME
108     : 'msec'
109     | 'sec'
110     | 'min'
111     | 'hour'
112     ;
113 BOOL: 'true' | 'false';
114 IDENTIFIER
115     : (LETTER | DIGIT | '_' | '-')+
116     ;
117 DIGIT: [0-9];
118 LETTER: [A-Z] | [a-z];
119 TEXT: [a-z]+;
120
121 WS: [ \t\n\r\f]+ -> skip ;

```

Appendix B

eShop I2 - Most Recent Price in Java

```
1 package org.myorg.flinkinvariants.invariantcheckers;
2
3 import com.fasterxml.jackson.databind.JsonNode;
4 import org.apache.flink.api.common.eventtime.WatermarkStrategy;
5 import org.apache.flink.cep.CEP;
6 import org.apache.flink.cep.functions.PatternProcessFunction;
7 import org.apache.flink.cep.pattern.Pattern;
8 import org.apache.flink.cep.pattern.conditions.IterativeCondition;
9 import org.apache.flink.cep.pattern.conditions.SimpleCondition;
10 import org.apache.flink.streaming.api.datastream.DataStream;
11 import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
12 import org.apache.flink.streaming.api.functions.sink.PrintSinkFunction;
13 import org.apache.flink.streaming.api.functions.sink.SinkFunction;
14 import org.apache.flink.streaming.api.windowing.time.Time;
15 import org.apache.flink.util.Collector;
16 import org.myorg.flinkinvariants.datastreamsourceproviders.KafkaReader;
17 import org.myorg.flinkinvariants.events.EShopIntegrationEvent;
18 import org.myorg.flinkinvariants.events.EventType;
19 import java.time.Duration;
20 import java.util.Iterator;
21 import java.util.List;
22 import java.util.Map;
23
24
25 public class ProductPriceChangedInvariantChecker {
26
27     private static final int MAX_LATENESS_OF_EVENT = 30;
28
29     public static void main(String[] args) throws Exception {
30         final StreamExecutionEnvironment env =
31             ↪ StreamExecutionEnvironment.getExecutionEnvironment().setParallelism(1);
32
33         var streamSource = KafkaReader.GetDataStreamSource(env)
34             .assignTimestampsAndWatermarks(WatermarkStrategy
35                 .<EShopIntegrationEvent>forBoundedOutOfOrderness(
36                     Duration.ofSeconds(MAX_LATENESS_OF_EVENT))
37                 .withIdleness(Duration.ofSeconds(2))
38                 .withTimestampAssigner((event, timestamp) -> event.getTimestamp()));
39         CheckProductPriceChangedInvariant(env, streamSource, new PrintSinkFunction<>());
40     }
41
42     public static void CheckProductPriceChangedInvariant(StreamExecutionEnvironment env,
43         DataStream<EShopIntegrationEvent> input,
44         SinkFunction<String> sinkFunction) throws Exception {
45         var filteredStream = input.filter(e ->
46             e.EventName.equals("UserCheckoutAcceptedIntegrationEvent")) ||
```



```

46         e.EventName.equals("ProductPriceChangedIntegrationEvent")).setParallelism(1);
47
48     var patternStream = CEP.pattern(filteredStream, InvariantPattern);
49     var matches = patternStream
50         .inEventTime()
51         .process(new PatternProcessFunction<EShopIntegrationEvent, String>() {
52             @Override
53             public void processMatch(Map<String, List<EShopIntegrationEvent>> map,
54                 Context context, Collector<String> collector) {
55                 collector.collect(map.toString());
56             }
57         })
58         .addSink(sinkFunction);
59     env.execute("Flink eShop Product Price Changed Invariant");
60 }
61
62 public static Pattern<EShopEvent, ?> InvariantPattern =
63     ↪ Pattern.<EShopEvent>begin("firstPriceChange")
64     .subtype(ProductPriceChangedEvent.class)
65     .notFollowedBy("subsequentPriceChange")
66     .where(new IterativeCondition<>() {
67         @Override
68         public boolean filter(EShopEvent pc2, Context<EShopEvent> context) throws Exception {
69             if (!pc2.EventName.equals("ProductPriceChangedIntegrationEvent"))
70                 return false;
71             for (var pc1 : context.getEventsForPattern("firstPriceChange")) {
72                 return pc1.EventBody.get("ProductId").asInt() ==
73                     ↪ pc2.EventBody.get("ProductId").asInt();
74             }
75             return false;
76         }
77     })
78     .followedByAny("userCheckoutWithOutDatedItemPrice")
79     .where(new IterativeCondition<>() {
80         @Override
81         public boolean filter(EShopEvent c, Context<EShopEvent> context) throws Exception {
82             if (!c.EventName.equals("UserCheckoutAcceptedIntegrationEvent"))
83                 return false;
84             for (var pc1 : context.getEventsForPattern("firstPriceChange")) {
85                 var affectedProductId = pc1.EventBody.get("ProductId").asInt();
86                 var newPrice = pc1.EventBody.get("NewPrice").asDouble();
87                 for (Iterator<JsonNode> it = c.EventBody.get("Basket").get("Items").elements();
88                     ↪ it.hasNext();) {
89                     var item = it.next();
90                     if (affectedProductId == item.get("ProductId").asInt()
91                         && newPrice != item.get("UnitPrice").asDouble())
92                         return true;
93                 }
94             }
95             return false;
96         }
97     })
98     }.within(Time.seconds(30));

```

Appendix C

eShop I1 - Overselling Items

```
1 package org.myorg.flinkinvariants.invariantcheckers;
2
3 import org.apache.flink.api.common.eventtime.WatermarkStrategy;
4 import org.apache.flink.api.common.functions.FilterFunction;
5 import org.apache.flink.api.common.functions.RichFlatMapFunction;
6 import org.apache.flink.api.common.state.ValueState;
7 import org.apache.flink.api.common.state.ValueStateDescriptor;
8 import org.apache.flink.configuration.Configuration;
9 import org.apache.flink.streaming.api.datastream.DataStream;
10 import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
11 import org.apache.flink.streaming.api.functions.sink.PrintSinkFunction;
12 import org.apache.flink.streaming.api.functions.sink.SinkFunction;
13 import org.apache.flink.table.api.Schema;
14 import org.apache.flink.table.api.Table;
15 import org.apache.flink.table.api.bridge.java.StreamTableEnvironment;
16 import org.apache.flink.util.Collector;
17 import org.myorg.flinkinvariants.datastreamsourceproviders.KafkaReader;
18 import org.myorg.flinkinvariants.events.EShopIntegrationEvent;
19 import org.myorg.flinkinvariants.events.EventType;
20 import java.io.IOException;
21 import java.time.Duration;
22
23 public class ProductOversoldInvariantChecker {
24
25     private static final int MAX_LATENESS_OF_EVENT = 30;
26     public static void main(String[] args) throws Exception {
27         final StreamExecutionEnvironment env =
28             ↪ StreamExecutionEnvironment.getExecutionEnvironment().setParallelism(1);
29         var streamSource = KafkaReader.GetDataStreamSource(env)
30             .assignTimestampsAndWatermarks(WatermarkStrategy
31                 .<EShopIntegrationEvent>forBoundedOutOfOrderness(
32                     Duration.ofSeconds(MAX_LATENESS_OF_EVENT))
33                 .withIdleness(Duration.ofSeconds(2))
34                 .withTimestampAssigner((event, timestamp) -> event.getTimestamp()));
35         CheckOversoldInvariant(env, streamSource, new PrintSinkFunction<>());
36     }
37
38     public static void CheckOversoldInvariant(StreamExecutionEnvironment env,
39         DataStream<EShopIntegrationEvent> input,
40         SinkFunction<String> sinkFunction
41     ) throws Exception {
42         StreamTableEnvironment tableEnv = StreamTableEnvironment.create(env);
43         Table table = tableEnv.fromDataStream(input, Schema.newBuilder()
44             .columnByMetadata("rowtime", "TIMESTAMP_LTZ(3)")
45             .watermark("rowtime", "SOURCE_WATERMARK()")
46             .build());
47         tableEnv.createTemporaryView("events", table);
48         // use table API to sort events by event time
49         Table sorted = tableEnv.sqlQuery("SELECT * FROM events ORDER BY rowtime ASC");
```

```

49     DataStream<EShopIntegrationEvent> sortedStream = tableEnv.toDataStream(sorted).map(r ->
50         (EShopIntegrationEvent) r.getFieldAs(0)).setParallelism(1);
51     var violations = sortedStream
52         .keyBy(r -> r.EventBody.get("ProductId"))
53         .flatMap(new OversoldMapper())
54         .addSink(sinkFunction).setParallelism(1);
55     env.execute("Flink eShop Product Oversold Invariant");
56 }
57
58 static class OversoldMapper extends RichFlatMapFunction<EShopIntegrationEvent, String> {
59     /** The state for the current key. */
60     private ValueState<Integer> currentStock;
61
62     @Override
63     public void open(Configuration conf) {
64         // get access to the state object
65         currentStock = getRuntimeContext().getState(
66             new ValueStateDescriptor<>("ProductStock", Integer.class));
67     }
68
69     @Override
70     public void flatMap(EShopIntegrationEvent event, Collector<String> collector) throws
71         ↪ Exception {
72         // get the current stock for the key (Product)
73         var productId = event.EventBody.get("ProductId").asText();
74         var eventType = EventType.valueOf(event.EventName);
75         switch (eventType) {
76             case ProductCreatedIntegrationEvent
77                 -> currentStock.update(event.EventBody.get("AvailableStock").asInt());
78             case ProductStockChangedIntegrationEvent
79                 -> currentStock.update(event.EventBody.get("NewStock").asInt());
80             case ProductDeletedIntegrationEvent
81                 -> currentStock.update(null);
82             case ProductBoughtIntegrationEvent
83                 -> HandleProductBoughtIntegrationEvent(event, collector, productId);
84         }
85     }
86
87     private void HandleProductBoughtIntegrationEvent(EShopIntegrationEvent event,
88         Collector<String> collector, String productId
89     ) throws IOException {
90         Integer stock = currentStock.value();
91         if (stock == null) {
92             collector.collect("Violation: Bought deleted Item with Id: " + productId);
93             return;
94         }
95         var unitsBought = event.EventBody.get("Units").asInt();
96         if (unitsBought < 0) {
97             collector.collect("Violation: units bought negative for ProductId: " + productId
98                 + ", units bought: " + unitsBought);
99             return;
100         }
101         if (stock < unitsBought) {
102             collector.collect("Violation: stock not sufficient for ProductId: " + productId
103                 + ", current stock: " + stock + ", units bought: " + unitsBought);
104         } else {
105             currentStock.update(stock - unitsBought);
106         }
107     }
108 }

```

Appendix D

Monitoring Dashboard



Figure D.1: Monitoring Dashboard